

## Project 1: Bayesian Structure Learning

**Tristan Krueger**

*AA228/CS238, Stanford University*

TKRUEGER@STANFORD.EDU

### 1. Algorithm Description

My implementation of the `compute()` function estimates the directed-acyclic graph (Bayesian Network) that would best fit a given provided CSV dataset using Bayesian score optimization.

After loading the data into a NumPy matrix, the program first runs the **K2** algorithm to search for the best-fit graph. The **K2** algorithm relies on an initial node ordering, which it uses to ensure that the graph is acyclic and directed. Since we have no prior for the optimal initial ordering, my code iterates through 100 random orders, running **K2** on each and choosing the best result.

Every run of **K2**'s fit algorithm begins with the initialization of a graph with no edges, with each data column representing a node. It then calculates an initial Bayesian score for the empty graph.

It then iterates through each node, starting at the second entry (since the first, highest-hierarchy entry cannot have any parents). It then iterates through each higher-hierarchy node, testing whether the addition of an edge from that node improves the Bayesian score. After each potential parent has been tested, we add the most-improving edge (if such an edge exists) and then continue to search for improving parent-edges until none can be found. It then moves to the next node and repeats the process on each in the graph.

After the 100 iterations of **K2** are complete, we move onto a second optimizing step based on **Local Search**. Normally this algorithm starts with a random graph and makes single edge operations to test if they improve the fit. However, since we already have the **K2** output, I start with this graph to see if any improvement can be made.

In the **Local Search** step, we iterate 10,000 times. For each, we choose a random neighbor of our current best-fit graph (*neighbor* here means that exactly one random edge was either: (1) created, (2) removed, or (3) reversed). We then test the Bayesian score of that neighbor. If it's better than our original's, the neighbor becomes our new best-fit graph.

During **Local Search**, it's possible to reach a state where no local operations can improve the graph. However, this does not necessarily mean that we have reached an optimal solution. Therefore, this code attempts to recognize when there has been such a stagnation

(10 consecutive operations which return the same score or 50 operations in a row without improving the score). When stagnation is found, we stop iterating on the current graph and, instead, test whether a series of randomly generated graph are superior to our current best-fit. To generate these random graphs, we start with a randomized, topological node order and add a random number of directed edges. It is important to note that cyclic graphs can be produced during **Local Search**; we check each graph and reject cyclic neighbor candidates.

The final, key piece of this program is the calculation of the Bayesian score based on the data and graph. To find the  $m_{ijk}$  counts of the dataset, I iterate through each node in topological order. For each, we slice the data matrix to only include values for the node and its parents. We then treat each sample as a coordinate of a multi-dimensional  $r_i \times r_{q_1} \times r_{q_2} \times \dots \times r_{q_n}$  matrix, using `ravel_multi_index()` to map these coordinates to a single dimension. We then, essentially, have a flattened  $r_i \times q_i$  matrix. We use `bincount()` to count the unique coordinates (samples) and then reshape the vector to the desired  $r_i \times q_i$  matrix, which gives us all  $m_{ijks}$ .

We then use these counts to compute the Bayesian score, which measures how well our graph models the provided dataset.

At 100 **K2** iterations and 10,000 **Local Search** iterations, the run which produced my submitted graphs took:

1. 1.55 seconds for `small.csv`;
2. 16.20 seconds for `medium.csv`; and
3. 1,492.22 seconds for `large.csv`.

## 2. Graphs

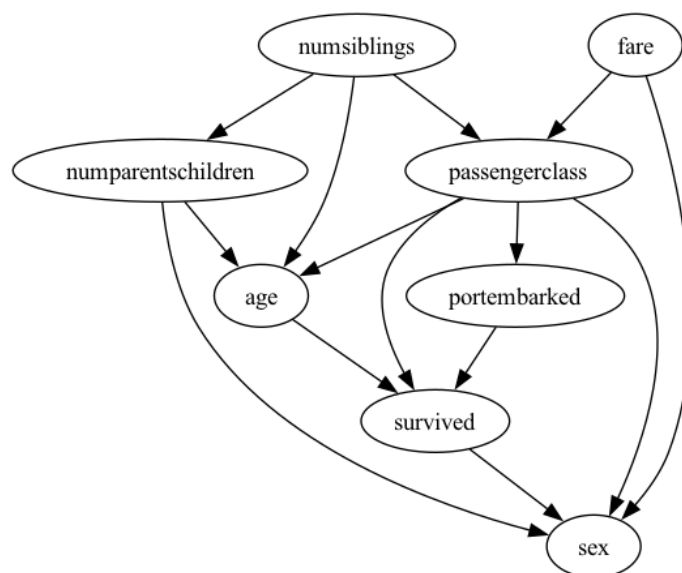


Figure 1: Small dataset graph.

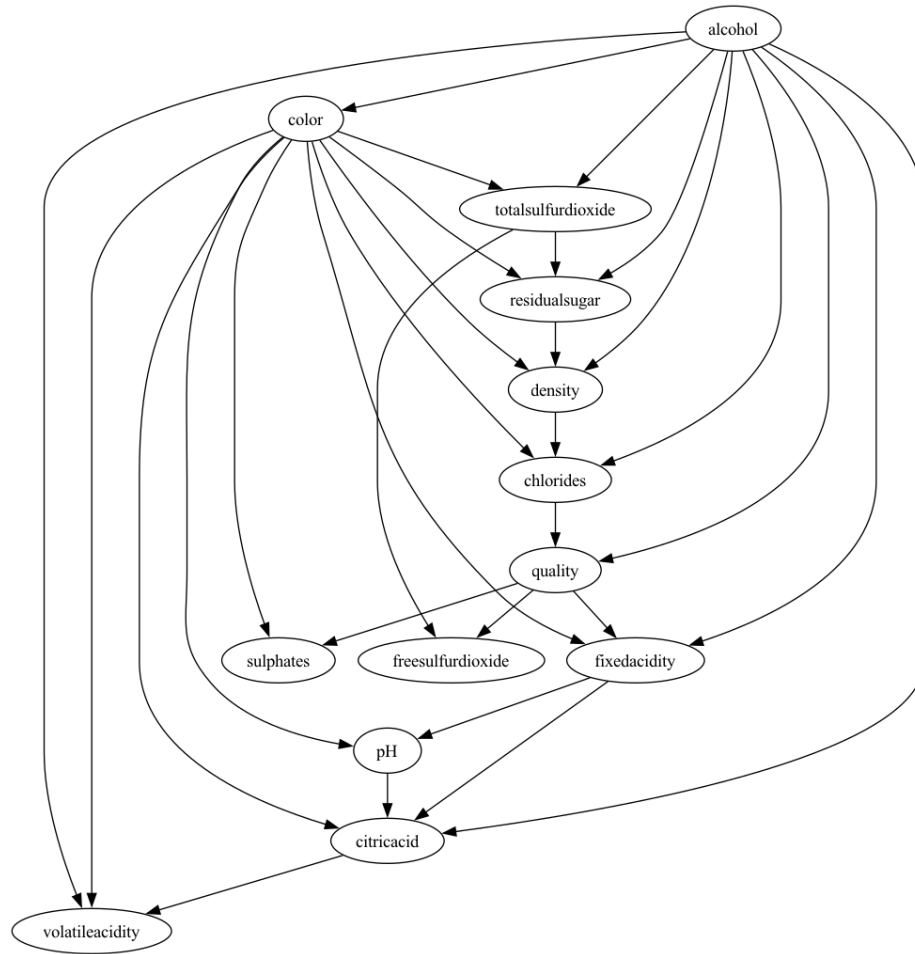


Figure 2: Medium dataset graph.

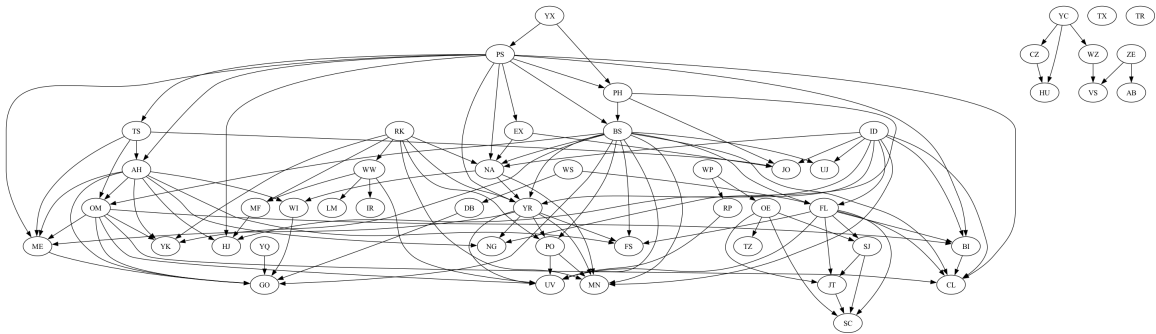


Figure 3: Large dataset graph.

### 3. Code

```

import sys
import numpy as np
import networkx as nx
from typing import Tuple
import math
import csv
import random
import scipy.special as sc
from pathlib import Path
import time

K2_ITERATIONS = 100
LOCAL_SEARCH_ITERATIONS = 10000

# python3 project1.py tiny.csv outfile.gph
def write_gph(G, node_names, filename):
    with open(filename, 'w') as f:
        for edge in G.edges():
            print((f'{node_names[edge[0]]}, {node_names[edge[1]]}'))
            f.write("{} {} \n".format(node_names[edge[0]], node_names[edge[1]]))

    # Add labels to graph for plotting
    for node in G.nodes():
        G.nodes[node]["label"] = node_names[node]

    dot_path = Path(filename).with_suffix(".dot")

    nx.nx_pydot.write_dot(G, dot_path)

def compute(infile: str, outfile: str, print_only=True):
    S = time.time()
    node_names, D = extract_raw_data(infile)

    # Run k2 to get a baseline
    G = iterative_k2(D, K2_ITERATIONS)

    # Perform additional local search optimization step to make marginal
    improvements
    G = local_search(G, D, LOCAL_SEARCH_ITERATIONS)

    print(f'Took {time.time() - S}s to run for {infile}.')

    if (print_only):
        for edge in G.edges():
            print((f'{node_names[edge[0]]}, {node_names[edge[1]]}'))
    else:
        write_gph(G, node_names, outfile)
    pass

```

```

def extract_raw_data(path: str) -> Tuple[np.ndarray, np.ndarray]:
    with open(path, mode='r', newline='', encoding='utf-8') as file:
        reader = csv.reader(file)
        data_list = list(reader)

    node_names = data_list[0]
    D = np.array(data_list[1:], dtype=np.uint8)

    return (node_names, D)

def get_graph_stats_efficiently(DG: nx.DiGraph, D: np.ndarray) -> np.ndarray:
    """
    Improved version to gather indexes and counts from sample-set D.

    This method took in a directed NetworkX graph and the dataset. It
    iterates through the nodes (X_is) and unique
    samples. For each, it fills m_ijk counts into a q_i x r_i matrix.
    Importantly, it iterates through the nodes
    in topological order so that parent counts are populated first.

    Unlike the earlier attempt - 'get_graph_stats()' - it does so without
    iterating through the samples. Specifically,
    it:
    1. Slices the D matrix to gather only the columns relevant to X_i
    2. Subtracts 1 from each value so that the values can be treated as
    indexes
    3. Treats each row of the slice as a coordinate in a r_i x r_q_1 x r_q_2
    x ... x r_q_n matrix. Uses
    'ravel_multi_index()' to map these n+1-dimensional coordinates to 1D
    vector coordinates.
    4. Counts the indices in the flattened matrix with 'bincount()'. Pads the
    matrix as needed with 0s.
    5. Reshapes the vector back to the desired q_i x r_i matrix.
    """
    ordered_nodes = list(nx.topological_sort(DG))
    stats = list(range(len(ordered_nodes)))

    n = len(ordered_nodes)

    # Create sparse matrix for value tracking
    R = np.ones(n, dtype=int)

    indexable_D = D - 1

    for node in ordered_nodes:
        parents = list(DG.predecessors(node))
        node_col = D[:, node]
        k_i = int(np.max(node_col))

```

```

    R[node] = k_i

    relevant_nodes = [node] + parents
    coordinates = []
    dimensions = []

    for i in relevant_nodes:
        coordinates.append(indexable_D[:,i])
        dimensions.append(R[i])

    flattened_node_data = np.ravel_multi_index(coordinates, dimensions)

    count_array = np.bincount(flattened_node_data)

    padding_size = np.prod(dimensions) - len(count_array)

    new_dimensions = [dimensions[0], int(np.prod(dimensions[1:])) or 1]

    stats[node] = np.pad(count_array, (0, padding_size), mode='constant')
    .reshape(new_dimensions)

    return stats

def get_graph_stats(DG: nx.DiGraph,
                    unique_samples: np.ndarray,
                    sample_counts: np.ndarray) -> list:

    """
    Initial version no longer used in codebase (left for visibility)

    This method took in a directed NetworkX graph and a uniquified dataset (
    along with the
    associated counts for each unique row). It iterated through the nodes (
    X_is) and unique
    samples to build a list of length n, each index filled with a q_i x r_i
    matrix.
    """
    ordered_nodes = list(nx.topological_sort(DG))
    stats = list(range(len(ordered_nodes)))

    n = len(ordered_nodes)

    # Create sparse matrix for value tracking
    R = np.ones(n, dtype=int)

    for node in ordered_nodes:
        parents = list(DG.predecessors(node))
        node_col = unique_samples[:, node]
        k_i = int(np.max(node_col))

```

```

    R[node] = k_i

    q_i = np.prod(R[parents], dtype=int)

    i_matrix = np.zeros((k_i, q_i))

    for index, sample in enumerate(unique_samples):
        k_index = sample[node] - 1
        parental_instance = sample[parents] - 1
        parental_sizes = R[parents]
        parental_instance_index = np.ravel_multi_index(parental_instance,
        parental_sizes)
        i_matrix[k_index][parental_instance_index] += sample_counts[index]
    ]

    stats[node] = i_matrix

    return stats

def compute_bayesian_score(M: np.ndarray) -> float:
    n = len(M)
    score = 0
    for i in range(n):
        k_i, q_i = M[i].shape
        a_ij0 = k_i
        m_ij0 = np.sum(M[i], axis=0)

        score += q_i * sc.loggamma(a_ij0)
        score -= np.sum(sc.loggamma((a_ij0 + m_ij0).astype(np.float64)))
        score += np.sum(sc.loggamma(1 + M[i].astype(np.float64)))

    return score

def bayesian_score_from_graph(G: nx.DiGraph, D: np.ndarray) -> float:
    M = get_graph_stats_efficiently(G, D)
    return compute_bayesian_score(M)

def iterative_k2(D: np.ndarray, n_orders: int) -> nx.DiGraph:
    n = D.shape[1]
    G_best = None
    score_best = -math.inf
    for _ in range(n_orders):
        rng = np.random.default_rng()
        ordered_nodes = rng.permutation(n)
        G, score = k2(ordered_nodes, D)
        if score > score_best:
            score_best = score
            G_best = G

```

```

print(f'Final Bayesian Score after K2: {score_best}\n')
return G_best

def k2(ordered_nodes: list, D: np.ndarray) -> Tuple[nx.DiGraph, float]:
    G = nx.DiGraph()
    G.add_nodes_from(ordered_nodes)

    final_score = -math.inf
    for (k,i) in enumerate(ordered_nodes[1:]):
        score = bayesian_score_from_graph(G, D)
        while True:
            score_best = -math.inf
            j_best = 0
            for j in ordered_nodes[0:k]:
                if not G.has_edge(j, i):
                    G.add_edge(j, i)
                    score1 = bayesian_score_from_graph(G, D)
                    if score1 > score_best:
                        score_best = score1
                        j_best = j
                    G.remove_edge(j, i)
            if score_best > score:
                score = score_best
                G.add_edge(j_best, i)
            else:
                final_score = score
                break

    return G, final_score

def random_cousin(G: nx.DiGraph) -> nx.DiGraph:
    """
    Given a NetworkX graph, create a new graph with the same nodes but
    with randomized edges.

    The newly created cousin is guaranteed to be directed-acyclic, since
    we enforce an edge
    direction for each new edge created.
    """
    G1 = nx.DiGraph()
    G1.add_nodes_from(G)
    n = len(G1.nodes())
    rng = np.random.default_rng()
    max_acyclic_edges = int((n * (n - 1)) / 2)
    min_attempted_edges = math.floor(n * .75)
    # Get a random number of edges between a chosen minimum (based on desired
    density) and the
    # maximum number of acyclic edges that the graph could support
    n_edges = random.randint(min_attempted_edges, max_acyclic_edges)

```

```

random_edges = np.unique(rng.integers(0, n, size=(n_edges, 2)), axis=0)

# Generate random node hierarchy
edge_order = rng.permutation(n)

for edge in random_edges:
    # Enfore edge direction
    edge = (edge[0],edge[1]) if edge_order[edge[0]] >= edge_order[edge
[1]] else (edge[1],edge[0])
    if not G1.has_edge(*edge):
        G1.add_edge(*edge)

return G1

def random_edge(n_nodes: int) -> Tuple[int, int]:
    i = random.randint(0, n_nodes - 1)
    remaining_options = list(range(n_nodes))
    del remaining_options[i]
    j = random.choice(remaining_options)
    return (i, j)

def local_search(G: nx.DiGraph, D: np.ndarray, iterations: int) -> nx.DiGraph
:
    score = bayesian_score_from_graph(G, D)
    stuck_count = 0
    stagnation_count = 0
    for _ in range(iterations):
        # If same value repeatedly or lack of improvement over time, generate
        a new random graph
        G1 = random_neighbor(G) if stuck_count < 10 and stagnation_count < 50
        else random_cousin(G)
        acyclic = nx.is_directed_acyclic_graph(G1)
        score1 = bayesian_score_from_graph(G1, D) if acyclic else -math.inf

        if score1 == score:
            stuck_count += 1
        else:
            stuck_count = 0

        if score1 > score:
            G = G1
            score = score1
            stagnation_count = 0
        else:
            stagnation_count += 1

    print(f'Final Bayesian Score after Local Search: {score}\n')
    return G

def random_neighbor(G: nx.DiGraph) -> nx.DiGraph:

```

```

n = len(list(G.nodes()))
i, j = random_edge(n)
G1 = G.copy()

if G.has_edge(i, j):
    G1.remove_edge(i, j)
    reverse_edge = random.randint(0, 1)
    if reverse_edge:
        G1.add_edge(j, i)
else:
    G1.add_edge(i, j)
return G1

def main():
    if len(sys.argv) < 3:
        raise Exception("usage: python project1.py <infile>.csv <outfile>.gph
")

    inputfilename = sys.argv[1]
    outputfilename = sys.argv[2]
    print_only = sys.argv[3] if len(sys.argv) > 3 else False

    compute(inputfilename, outputfilename, True if print_only else False)

if __name__ == '__main__':
    main()

```

#### 4. AI Disclosure

I try to make very limited use of AI when coding for classwork, as I find that it significantly hinders my learning. However, I did consult AI on two specific points:

1. After completing my initial implementation of the  $m_{ijk}$  index counting - `get_graph_stats()` - I ran into significant efficiency issues with the *large.csv*. It initially took around 30 minutes for a single **K2** run. I explained my implementation to ChatGPT and requested high-level ideas for how to more effectively vectorize my solution. It suggested using `np.bincount()`, which unblocked my optimization progress (I researched and coded the updated solution myself).
2. I also asked for suggestions regarding how to ensure that a randomly created graph would be acyclic. It suggested enforcing a topological hierarchy when creating new edges so that newly created random graphs would be directed-acyclic. I independently implemented this approach.